

Rapport de Projet

Chronomots



Auteurs : Danny MITH, Lamia BERCHICHE

2025-2026

Sommaire :

I. Introduction et Architecture

- Objectifs du projet Chronomots.
- Arborescence et rôles des fichiers.

II. Méthodologie et Conception

- Choix des structures et algorithmes (tris fusion, rapide, bulle).
- Justification des exclusions (table de hachage).
- Phases de développement : prototypage, modularité et tests.

III. Étude Technique

- Implémentation des listes (simples/doubles) et tableaux dynamiques.
- Analyse des fonctions clés et de la complexité théorique.

IV. Analyse des Performances

- Protocole d'automatisation et de visualisation (Python).
- Comparaison des temps d'exécution et de l'occupation mémoire.
- Analyse des écarts selon la machine et la gestion de l'OS.

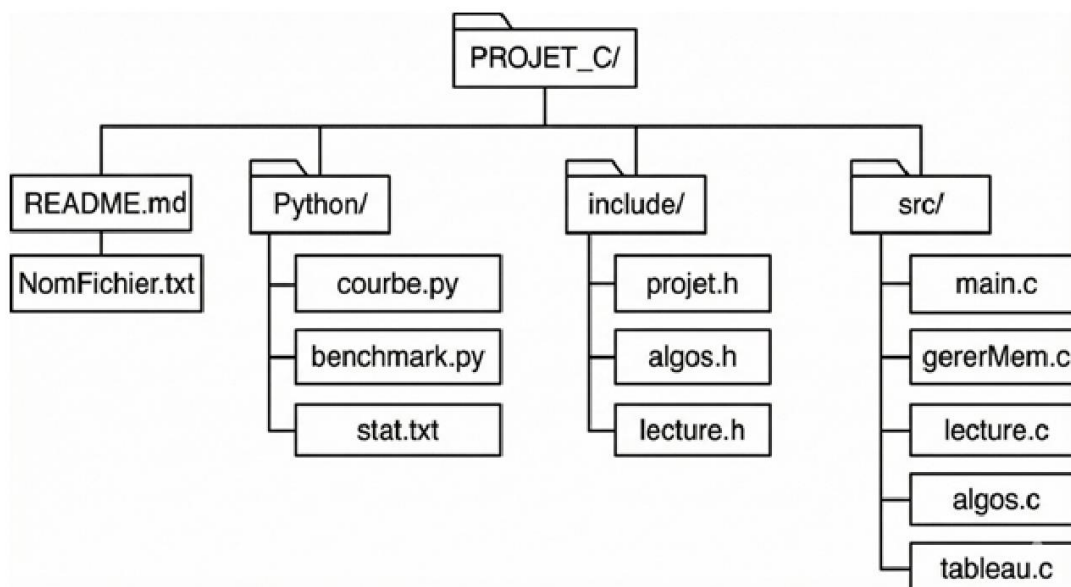
V. Bilan et Améliorations

- Répartition des tâches et journal de bord.
- Fonctionnalités avancées (filtres, mots bannis).
- Conclusion : adéquation algorithmes-structures.

Introduction/but :

L'objectif est de réaliser un programme qui recense les mots contenus dans un fichier texte, en utilisant différents algorithmes et structures de données, et en comparant les résultats obtenus en termes d'efficacité.

Arborescence des fichiers :



Rôles de chaque fichier :

- | | |
|------------------|--|
| 📄 README.md | # Guide d'utilisation et compilation. |
| 📄 NomFichier.txt | # Fichiers textes (ex: livre.txt) |
| 📁 Python | |
| 📄 courbe.py | # Génération visuelle des graphiques. |
| 📄 benchmark.py | # Automatisation des tests de performance. |
| 📄 stat.txt | # Sauvegarde des performances |
| 📁 include | |
| 📄 projet.h | # Les fichiers .h (prototypes) |
| 📄 algos.h | # Structures globales (InfoMem, etc.) |
| 📄 lecture.h | # Prototypes des fonctions de tri/comptage |
| | # Prototypes du traitement de texte. |

src	# Les fichiers .c (code source)
main.c	# Analyse des arguments et appel principaux
gererMem.c	# Gestionnaire personnalisé de mémoire
lecture.c	# Analyse et découpage du texte source.
algos.c	# Implémentation des algorithmes sur listes
tableau.c	# Implémentation des algorithmes sur tableaux

Organisation et méthodologie de recherche :

Dès la découverte du sujet, l'utilisation des **listes chaînées** s'est imposée comme une évidence, cette structure ayant été étudiée en détail durant le semestre. Concernant l'aspect algorithmique, le **tri fusion** abordé en L1 puis lors d'un TP de L2 constituait également un choix naturel et cohérent.

Après avoir développé une première version fonctionnelle basée sur ces éléments, nous avons souhaité explorer d'autres pistes afin d'enrichir notre projet. Une phase de réflexion nous a permis d'établir une liste de structures et d'algorithmes potentiels, avant d'opérer une sélection basée sur leur faisabilité :

Algorithmes envisagés :

- Tri à bulles
- Tri rapide (Quicksort)
- Tri fusion
- Tri par insertion

Structures de données envisagées :

- Tableau dynamique
- Table de hachage
- Liste doublement chaînée

Justification des choix et exclusions

Certaines options n'ont pas été retenues pour des raisons de temps ou de complexité d'implémentation. C'est notamment le cas de la **table de hachage** : bien qu'elle soit théoriquement très performante avec une complexité en $O(n)$ amortie lors de la recherche, nous avons choisi de ne pas l'implémenter. En effet, la conception d'une fonction de hachage robuste est complexe, et nous avons préféré privilégier la fiabilité et la stabilité du programme plutôt que de risquer une implémentation "bancale" qui aurait pu pénaliser l'ensemble du projet.

Quant au choix final des algorithmes, nous avons privilégié l'**efficacité théorique**. C'est pour cette raison que le tri par insertion a été écarté au profit d'algorithmes plus performants, afin de garantir une exécution optimale sur de gros volumes de données.

Processus de conception :

1. Approche conceptuelle et prototypage Une étape préliminaire non négligeable a consisté à rédiger les prototypes des fonctions en français (pseudo-code). Cette méthode nous a permis de bien appréhender les problématiques algorithmiques avant la phase de codage. Pour la conception, nous nous sommes appuyés sur nos acquis (notamment ceux vus en TP et TD de L1 et L2) pour l'implémentation des algorithmes de tri. Le défi n'était pas de réinventer l'algorithme, dont la logique était déjà connue, mais de l'adapter efficacement aux structures de données spécifiques que nous avons utilisées.

2. Architecture et modularité Préalablement à l'implémentation

Nous avons segmenté le projet en plusieurs fichiers. Deux axes majeurs s'en dégagent : la gestion de la mémoire et l'optimisation de la rapidité. Une fois cette architecture établie, nous avons développé les fonctions selon un découpage pratique pour garantir la lisibilité du code.

Ce processus a été itératif ; plusieurs versions ont été nécessaires avant d'aboutir à la structure finale. Par exemple, nous avons d'abord isolé la fonction de lecture du texte, puis celle dédiée au stockage des mots, et enfin celle chargée du traitement des caractères spéciaux.

3. test et validation

Les tests ont été réalisés de manière progressive, en commençant par des fichiers de taille réduite (par exemple, des textes ne contenant que deux mots différents avec des occurrences variables). Cela nous a permis de vérifier la validité de notre logique algorithmique de base. Une fois les résultats fonctionnels confirmés, nous avons pu passer à l'étape suivante : la gestion rigoureuse de la mémoire et le passage à l'échelle sur de plus gros volumes de données.

Étude Technique des Algorithmes

Structures de données :

Nous avons un type « Mot » qui permet de stocker un mot sous forme de chaîne de caractère, ainsi que son nombre d'occurrence ;

Nous avons mis en œuvre 3 structures pour stocker ce type :

- **Liste chaînée** : ou chaque nœud pointe vers le mot suivant
- **Liste doublement chaînée** : semblable à la structure précédente, avec un pointeur en plus qui permet en plus de revenir en arrière

- **Tableau dynamique** : gérer par réallocation dynamique pour minimiser les couts

Prototypes des fonctions clés :

- *Char* lire_mot_dynamique(FILE *f, InfoMem *mem)*

Cette fonction extrait un mot unique du flux de fichier passé en paramètre. Elle alloue la mémoire caractère par caractère et utilise un redimensionnement dynamique (via myRealloc) pour s'adapter précisément à la taille du mot lu, garantissant ainsi une utilisation optimale de la mémoire vive.

- *saisir_mots_bannis(ListeBan *liste, int n)*

Permet de remplir une structure de données contenant les "mots bannis" (à ignorer lors de l'analyse).

- *triFusion(Liste *lst, int est_double)*

Implémente l'algorithme de tri "Diviser pour Régner" avec une complexité temporelle de $O(n \log(n))$. La fonction divise récursivement la liste en deux via diviserListe puis les fusionne. Le paramètre *est_double* permet d'activer la mise à jour des pointeurs précédent pour maintenir l'intégrité des listes doublement chaînées.

- *tri_bulle(Liste *lst, int est_double)*

Algorithme de tri par comparaison de complexité $O(n^2)$. Il parcourt la liste de manière répétée et échange les structures Mot (données) entre deux nœuds adjacents si elles ne sont pas dans l'ordre défini par *est_mieux_place*. Le paramètre *est_double* est ici utilisé pour adapter l'algorithme selon la structure de données utilisées (liste chaînée simple/double)

- *ajouter_dans_liste(Liste *tete, char *nouveau_mot, InfoMem *mem, int est_double)*

Cette fonction gère l'insertion et le comptage des mots. Elle parcourt d'abord la liste pour vérifier si le mot existe déjà ; si c'est le cas, elle incrémente le compteur d'occurrences et libère la mémoire du mot lu en double. Sinon, elle crée un nouveau nœud, l'insère en tête, et gère le lien retour précédent si le mode *est_double* est activé.

Complexité théorique

- **Tri Fusion (*triFusion*)** : Utilisé principalement sur les listes. Il repose sur le principe "Diviser pour régner" avec une fonction de fusion combinée qui gère

aussi bien les listes simples que doubles. Sa complexité théorique est de $O(n \log(n))$

- **Tri Rapide (*tri_rapide_tab*)** : Implémenté pour la structure Tableau, offrant d'excellentes performances empiriques en $O(n \log(n))$
- **Tri à Bulles (*tri_bulle*)** : Implémenté à des fins comparatives. Sa complexité en $O(n^2)$ sert de point de référence pour illustrer l'inefficacité sur de gros volumes de données.
- **Note sur la comparaison** : Pour chaque algorithme, nous utilisons la fonction `est_mieux_place` qui trie par nombre d'occurrences décroissant, puis par ordre alphabétique en cas d'égalité afin d'obtenir le même ordre d'occurrence à chaque fois.

5. Protocole de Test et Automatisation

Le script **benchmark.py** automatise le lancement du programme C sur différents fichiers avec des combinaisons variées d'algorithmes et de structures. Le programme C enregistre ensuite les résultats dans `stats.txt` au format suivant :

[Nom_Fichier--AlgoStructure--Temps_Exec--Memoire_Max]

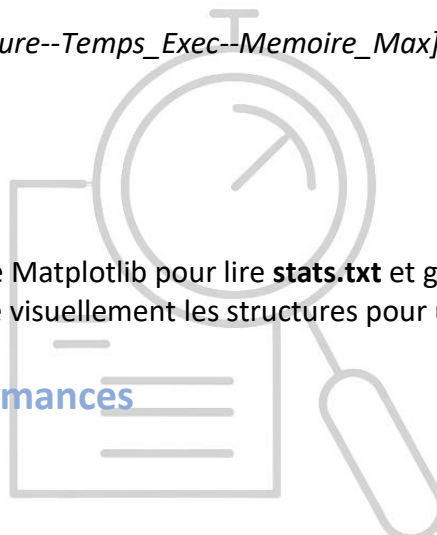
Format de sortie : .txt

Script de visualisation :

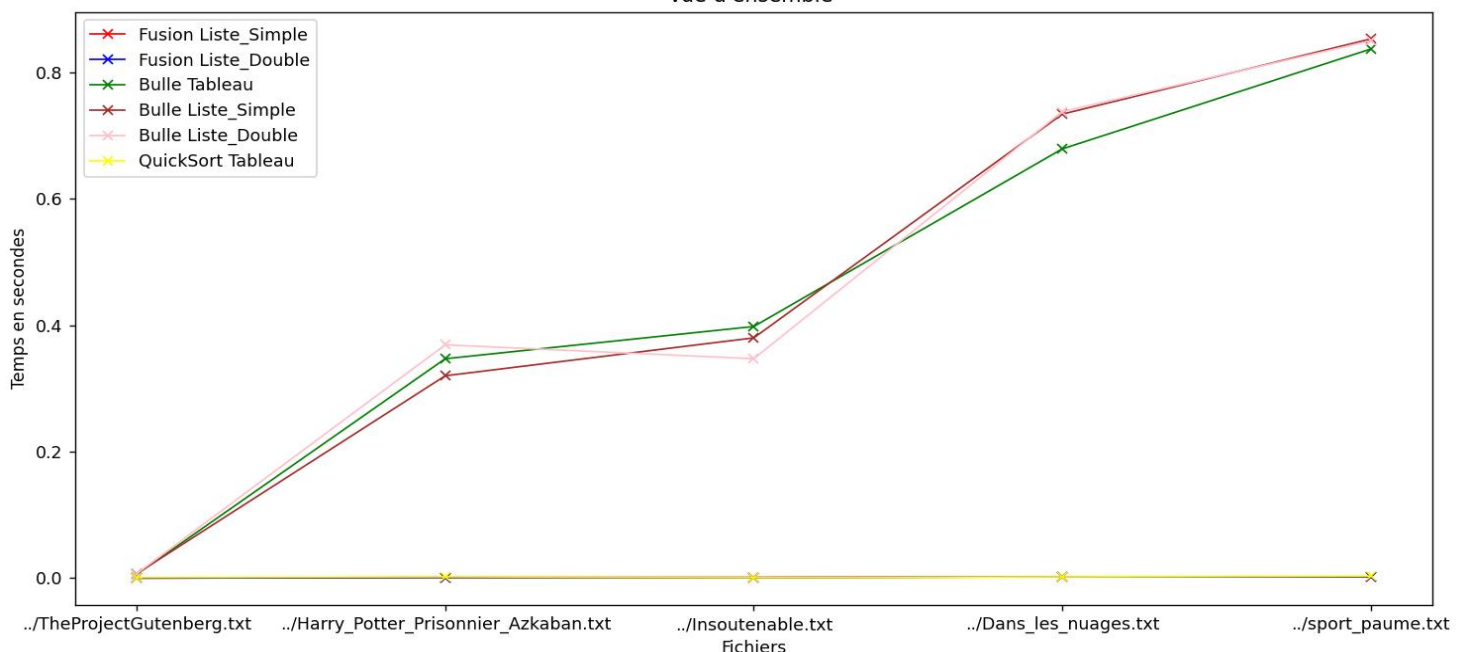
Le script **courbe.py** utilise Matplotlib pour lire `stats.txt` et générer des graphiques comparatifs. Il différencie visuellement les structures pour une comparaison immédiate.

Analyse des Performances

Courbe représentative :



Vue d'ensemble



Comparaison sur le tri a bulle :

Fichier Source	Tableau (Tps / RAM)	Liste Simple (Tps / RAM)	Liste Double (Tps / RAM)
Gutenberg	0.007s / 28 221 o	0.007s / 51 407 o	0.007s / 51 407 o
Harry Potter	0.325s / 671 055 o	0.320s / 356 099 o	0.319s / 356 099 o
Insoutenable	0.343s / 500 369 o	0.327s / 359 921 o	0.332s / 359 921 o
Dans les nuages	0.595s / 605 667 o	0.594s / 471 489 o	0.590s / 471 489 o
Sport Paume	0.705s / 422 869 o	0.733s / 502 280 o	0.732s / 502 280 o

Observation : la structure Tableau est parfois plus lente que les listes (ex: Dans les nuages), car les échanges de données dans un tableau peuvent être plus coûteux que le simple passage au pointeur suivant.

Comparaison sur le tri fusion et quick sort (tri pivot) :

fichier txt	Fusion (Liste Simple)	Fusion (Liste Double)	QuickSort (Tableau)
Gutenberg	0.000s / 51 407 o	0.000s / 51 407 o	0.000s / 28 221 o
Harry Potter	0.002s / 356 099 o	0.002s / 356 099 o	0.002s / 671 055 o
Insoutenable	0.001s / 359 921 o	0.002s / 359 921 o	0.001s / 500 369 o
Dans les nuages	0.001s / 471 489 o	0.002s / 471 489 o	0.002s / 605 667 o
Sport Paume	0.002s / 502 280 o	0.002s / 502 280 o	0.002s / 422 869 o

Observation : La Liste Simple avec le Tri Fusion rivalise souvent avec le QuickSort en termes de temps, tout en étant plus économe en mémoire RAM sur les gros fichiers comme Harry Potter.

Comparaison sur les meilleures performances de chaque structure :

Structure	Meilleur Algo	Temps Moyen observé
Tableau	QuickSort	~0.001s
Liste Simple	Fusion	~0.0012s
Liste Double	Fusion	~0.0016s

Analyse Comparative :

- **La Liste Simple + Fusion** est la solution la plus équilibrée : elle est presque aussi rapide que le QuickSort tout en utilisant souvent moins de mémoire maximale sur les fichiers à forte densité de mots.
- **Le Tableau + QuickSort** est imbattable sur les petits fichiers (50% de RAM en moins sur Gutenberg), mais sa consommation mémoire peut parfois s'envoler à cause du redimensionnement (realloc).
- **L'échec du Tri à Bulle** est sans appel : peu importe la structure, il est systématiquement 300 à 700 fois plus lent que les autres méthodes.
- **L'égalité parfaite** : Liste Simple vs Liste Double. On observe que pour chaque fichier testé, la mémoire consommée est strictement identique entre la Liste Simple et la Liste Double (Qui s'explique par la création du pointeur 'précédent' dans tous les cas). Ce qui annule tout gain de place théorique entre ces deux variantes.

Synthèse comparative par type de machine:

Une différence notable de performance a été constatée lors du lancement du programme sur nos deux machines respectives. En effet, nos environnements de test ne présentent pas les mêmes capacités techniques, l'une des machines étant nettement plus puissante que l'autre.

Fichier (Taille/Mots)	Algorithme / Structure	Machine 1 (Rapide)	Machine 2 (Lente)	Écart (Facteur)
Harry Potter	Bulle / Liste Simple	0.320 s	0.945 s	x 2.9 plus lent
Dans les nuages	Bulle / Tableau	0.595 s	2.089 s	x 3.5 plus lent
Sport Paume	Bulle / Liste Double	0.732 s	2.257 s	x 3.1 plus lent

Complexité théorique vs empirique

→ En termes de vitesse d'exécution :

- **Théorie** : On attend une différence massive de croissance. Le Tri Fusion et le QuickSort $O(n \log(n))$ doivent rester stables, tandis que le Tri à Bulle $O(n^2)$ doit exploser dès que la taille du fichier augmente.
- **Empirique** : Les mesures confirment strictement cette hypothèse. Sur le fichier sport_paume.txt, le Tri Fusion s'exécute en 0.002s alors que le Tri à Bulle nécessite 0.732s, soit un facteur de ralentissement de 366.

→ En termes de consommation mémoire :

- **Théorie** : Une liste doit consommer plus qu'un tableau car elle stocke des pointeurs (suivant, précédent) en plus des données.
- **Empirique** : Pour TheProjectGutenberg.txt, le tableau utilise 28 221 octets contre 51 407 pour les listes.
- **Explication** : Cet écart de ~45% correspond au fait que dans les listes : chaque mot nécessite la création d'un nœud qui pèse lourd par rapport à la donnée brute.

→ Exemple contre-intuitif :

- **L'écart surprenant** : Sur Harry_Potter.txt, le tableau monte à 671 055 octets alors que la liste reste à 356 099 octets.
- **Analyse** : C'est ici que la complexité empirique diverge de la théorie à cause de la gestion mémoire de l'OS. Pour optimiser les performances, votre tableau dynamique utilise probablement une stratégie de doublement de capacité.
- **Conséquence** : Si le tableau dépasse sa limite, il réserve immédiatement le double de place. Si le fichier s'arrête juste après cette extension, une grande partie de la mémoire Max utilise rapportée est en réalité du "vide" alloué mais non utilisé. À l'inverse, la liste chaînée alloue exactement ce dont elle a besoin, nœud par nœud, ce qui la rend parfois plus économe en pic de consommation sur les gros fichiers.

Bilan du Binôme

Répartition des tâches :

Membre : Danny MITH

Algorithmes : Tri à bulle

Contribution globale 50%.

Membre : Lamia BERCHICHE

Algorithmes : Tri fusion

Contribution globale 50%.

Journal de bord :

Par Danny MITH

- Gestion algorithme sur tableau et liste
- Automatisations des tests et affichage des courbes (Matplotlib) avec Python
- Gestion lecture (mots bannis)
- Implémentation de la structure tableau

Par Lamia BERCHICHE

- Gestion algorithme globale (liste chaînée)
- Gestion lecture
- Implémentation de la structure liste chaînée (double)
- Rédaction du rapport

Communs

- Main
- Implémentation de la structure liste chaînée (simple)
- Gestion mémoire
- Améliorations

Améliorations :

→ Les améliorations apportées sont les suivantes :

- **Nombre de mots** : possibilité de choisir le nombre de mots qu'on veut afficher.
- **Mots bannis** : possibilité d'ignorer une liste de mots interdits lors de la lecture du fichier.
- **Mots spécifiques** : possibilité de fournir une liste de mots précis à chercher.
- **Taille des mots** : filtrage par longueur (ex : extraire uniquement les mots de 5 lettres et plus).
- **Structures et algorithmes** : implémentation de 3 structures et de 2 algorithmes de tri (Bulle, Fusion) le Quick Sort étant une fonction de la bibliothèque standard.

Conclusion

L'analyse comparative menée durant ce projet met en évidence que l'efficacité d'un programme ne dépend pas uniquement de l'algorithme choisi, mais de son adéquation avec la structure de données utilisée.

Le **tableau dynamique** s'est révélé être la structure la plus performante, surpassant les listes chaînées (simples ou doubles).